COMPUTER LABORATORY 1

DATA AGGREGATION

Retail Sales Performance Analysis by Region

What this guide covers
Your exact Jupyter Notebook • line-by-line code explanation • Pandas and Matplotlib concepts • theory cross-questions • output interpretation • viva practice

Experiment objective

Analyze retail transactions to find total sales in each region and the contribution of each product category. Present the results using bar charts and save the aggregated tables as CSV files.

Prepared from the uploaded Assignment_12.ipynb notebook. The assignment list describes this as Data Aggregation: sales performance by region in a retail company (assignment-list page 5, item 12).

Simple explanations • Practical viva preparation • 9-page study guide

1. Understand the experiment

In a retail company, every row in a sales dataset usually represents a transaction or a product line in an order. The company may want to know which region earns the highest sales, and which product category contributes most in each region.

Data aggregation means combining many detailed records into a smaller summary using operations such as sum, count, average, minimum or maximum. In this notebook, the main aggregation is the sum of the Sales column.

The notebook uses the following workflow:

1. Load Read CSV	**2. Inspect** Preview & dtype	**3. Clean** Convert & trim	**4. Aggregate** Group and sum	**5. Visualize** Bar charts	**6. Export** Save CSV

Dataset columns used

The preview in your notebook shows 21 columns, including Order ID, Order Date, Customer Name, Region, Category, Sub-Category, Sales, Quantity, Discount and Profit. The analysis specifically uses **Region**, **Category** and **Sales**.

Expected business questions

• Which region has the highest total sales?

• Which region has the lowest total sales?

• How much does each category contribute within a region?

• How can the results be communicated clearly to a manager?

Important distinction: Sales is the revenue-like amount being summed here. Profit is a separate column and is not used in the aggregation performed by this notebook.

The notebook output shows West as the highest-sales region (725,457.8245) and South as the lowest (391,721.9050). These are sales totals, not profit totals.

2. Code walkthrough — setup and loading

Cell 1: Import libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path
```

import loads a library so its functions can be used in the program. **as** creates a shorter alias.

pandas (pd): used for tabular data. It provides DataFrame and Series structures, CSV reading, cleaning, grouping and exporting.

numpy (np): numerical computing library. It is imported in this notebook, but no NumPy function is actually called in the visible code. If asked, say it is available for numerical operations but unused in this experiment.

matplotlib.pyplot (plt): plotting interface used to draw bar charts.

Path: a pathlib class used to work with file and folder paths in a platform-friendly way.

Cell 2: Read the CSV

```
sales = pd.read_csv('Retail_Sales_Data.csv')
sales.head()
```

pd.read_csv() reads a comma-separated file and returns a Pandas DataFrame. The result is stored in the variable **sales**. The file must be in the notebook's current working directory, unless a correct path is provided.

sales.head() displays the first five rows by default. It is a quick preview, not a command that deletes or modifies rows. You can request a different number, for example sales.head(10).

Cell 3: Check the Region data type

```
sales['Region'].dtype
```

sales['Region'] selects one column as a Pandas Series. **.dtype** reports its data type. The notebook output is dtype('O'), where O means object. In this dataset, Region contains text labels such as Central, East, South and West.

Viva tip: A DataFrame is like a complete spreadsheet; a Series is like one column from that spreadsheet.

3. Code walkthrough — cleaning and aggregation

Cell 4: Clean the fields used

```
sales['Sales'] = pd.to_numeric(sales['Sales'], errors='coerce')
sales['Region'] = sales['Region'].str.strip()
sales['Category'] = sales['Category'].str.strip()
```

pd.to_numeric() attempts to convert values into numeric form. The argument **errors='coerce'** converts invalid values into NaN (Not a Number) instead of stopping with an error.

.str.strip() removes whitespace at the beginning and end of each text value. For example, 'West ' becomes 'West'. This matters because 'West' and 'West ' could otherwise be treated as two different groups.

Cell 5: Region-wise total sales

```
# region wise sales
region_sales = sales.groupby('Region')['Sales'].sum()
display(region_sales)
```

Line-by-line:

• **# region wise sales** is a comment. Python ignores it; it documents the purpose.

• **sales.groupby('Region')** divides rows into groups with the same Region value.

• **['Sales']** selects the Sales column from each group.

• **.sum()** adds the Sales values in each region.

• **region_sales** stores the resulting Pandas Series.

• **display()** shows the result neatly in a Jupyter Notebook.

The notebook output is:

Region	Total Sales
Central	501,239.8908
East	678,781.2400
South	391,721.9050
West	725,457.8245

Interpretation: West has the highest total sales, followed by East, Central and South. This ranking is based only on the sum of the Sales column.

4. Region + category aggregation and charts

Cell 6: Aggregate using two grouping columns

```
# region + category wise sales
reg_cat_sales = sales.groupby(['Region', 'Category'])['Sales'].sum().unstack(fill_value=0)
display(reg_cat_sales)

reg_cat_sales.plot(kind='bar', figsize=(13,4), stacked=True, rot=0)
plt.plot()
```

Here, **groupby(['Region', 'Category'])** creates groups for every unique Region–Category combination. For example, West + Furniture is one group and West + Technology is another. **.sum()** calculates the total Sales for each combination.

.unstack(fill_value=0) reshapes the grouped result: the inner grouping level (Category) becomes columns. Each row represents a Region and each category becomes a separate column. Missing combinations are filled with 0.

Your output table:

Region	Furniture	Office Supplies	Technology
Central	163,797.1638	167,026.415	170,416.312
East	208,291.2040	205,516.055	264,973.981
South	117,298.6840	125,651.313	148,771.908
West	252,612.7435	220,853.249	251,991.832

Chart 1: region-wise bar chart

```
region_sales.plot(kind='bar', figsize=(13,4), color='blue')
plt.plot()
```

kind='bar' requests vertical bars. **figsize=(13,4)** sets figure size in inches. **color='blue'** sets bar colour. Region names appear along the x-axis and total sales on the y-axis.

Chart 2: stacked bar chart

stacked=True places category values on top of one another for each region, so the total bar height represents total sales and the coloured segments show category contributions. **rot=0** keeps x-axis labels horizontal.

Note: The notebook calls plt.plot() after the Pandas plot. The usual Matplotlib command to display a chart in a normal Python script is plt.show(); Jupyter often displays the chart automatically. plt.plot() itself is not needed to display this bar chart.

5. Saving results + core theory

Cell 7: Create output folder and export

```
Path('output').mkdir(exist_ok=True)
region_sales.to_csv('output/region_sales.csv')
reg_cat_sales.to_csv('output/reg_cat_sales.csv')
```

Path('output') represents a folder named output. **.mkdir()** creates that folder. **exist_ok=True** means no error is raised if the folder already exists.

Series.to_csv() and **DataFrame.to_csv()** write the results into CSV files. The first file stores region totals; the second stores the region-by-category summary. These are saved relative to the current working directory.

Theory concepts to know

1. DataFrame: Two-dimensional labelled table with rows and columns. Example: sales.

2. Series: One-dimensional labelled data. Example: sales['Region'] or region_sales.

3. Data cleaning: Detecting and correcting missing, invalid, duplicated or inconsistent data before analysis.

4. Data aggregation: Summarizing detailed rows into group-level results using sum, mean, count, min or max.

5. GroupBy: Pandas split–apply–combine approach: split data into groups, apply an operation, combine results.

6. Missing values (NaN): Values that are absent or invalid. With errors='coerce', invalid numeric strings become NaN. Be aware that sum() normally skips NaN values; consider checking missing values before relying on totals.

7. CSV: Comma-Separated Values, a simple text format for tabular data. It is widely supported but does not preserve all spreadsheet formatting or data types.

8. Visualization: Representing data graphically to reveal comparisons, patterns and differences.

Aggregation vs transformation: Aggregation reduces each group to a summary; transformation generally returns data aligned to the original rows.

6. Functions used — viva cross-questions

Q. What is pandas?

Answer: A Python library for handling structured data. It provides Series and DataFrame objects and functions for reading, cleaning, grouping and exporting data.

Q. Why do we write import pandas as pd?

Answer: To import Pandas and use the short alias pd instead of typing pandas every time.

Q. What does read_csv() return?

Answer: Usually a Pandas DataFrame containing the rows and columns read from the CSV file.

Q. What is head() used for?

Answer: To preview the first five rows by default. head(n) previews the first n rows.

Q. What does dtype show?

Answer: It shows the data type of a Series or column. Object commonly represents text or mixed Python objects.

Q. What does to_numeric() do?

Answer: It converts values to numeric types where possible.

Q. Why errors='coerce'?

Answer: Invalid values become NaN instead of causing a conversion exception.

Q. What is str.strip()?

Answer: It removes leading and trailing whitespace from strings.

Q. What does groupby() do?

Answer: It groups rows that share the same value or combination of values in selected columns.

Q. What is sum() doing here?

Answer: It adds the Sales values inside each group.

Q. What is unstack()?

Answer: It pivots an index level into columns, changing the shape of a grouped result.

Q. Why fill_value=0?

Answer: It places zero where a Region–Category combination has no recorded value in the reshaped result. Zero means no recorded amount for that combination in this summary.

Q. What does plot(kind='bar') do?

Answer: It creates a bar chart from the Series or DataFrame.

Q. What is figsize=(13,4)?

Answer: It sets the figure width to 13 inches and height to 4 inches.

Q. What does stacked=True mean?

Answer: It stacks category bars within each region rather than drawing them side by side.

Q. What does rot=0 do?

Answer: It rotates x-axis tick labels by zero degrees, keeping them horizontal.

Q. What is Path from pathlib?

Answer: A class for representing and manipulating filesystem paths.

Q. What does mkdir(exist_ok=True) do?

Answer: Creates the directory if needed and does not fail merely because it already exists.

Q. What does to_csv() do?

Answer: It exports a Series or DataFrame to a CSV file.

Q. Why is NumPy imported if not used?

Answer: It is imported in the setup, but the shown notebook cells do not call NumPy. Do not claim it performs the aggregation.

7. Theory and concept cross-questions

Q. Why is data aggregation useful in business?

Answer: It converts thousands of transaction rows into understandable summaries, helping compare regions, categories and performance.

Q. What is the difference between sum and mean?

Answer: Sum adds all values; mean calculates the arithmetic average (sum divided by count of valid values).

Q. What is the difference between groupby one column and two columns?

Answer: One column creates groups by one field, such as Region. Two columns create groups by each unique pair, such as Region and Category.

Q. Why clean data before grouping?

Answer: Inconsistent labels can split one logical group into multiple groups; invalid numeric values can prevent correct calculations.

Q. What happens if Region contains 'West' and ' West '?

Answer: Without trimming, they may be treated as separate groups. strip() removes the extra outer space.

Q. What is an outlier?

Answer: A value unusually far from most other observations. It should be investigated; it is not automatically an error.

Q. What is the difference between missing and zero?

Answer: Missing means no value is recorded or known; zero is a real numeric value. They should not be treated as interchangeable without a business reason.

Q. What is a bar chart best for?

Answer: Comparing quantities across discrete categories, such as total sales by region.

Q. What is a stacked bar chart best for?

Answer: Showing both the total for each group and how its subcategories contribute to that total.

Q. Stacked vs grouped bar chart?

Answer: Stacked bars place series on top of one another; grouped bars place them side by side for direct category comparison.

Q. What is a pie chart used for?

Answer: Showing parts of a whole, usually when there are a small number of categories.

Q. What is data wrangling?

Answer: The process of cleaning, structuring and transforming raw data into a form suitable for analysis.

Q. What is a CSV file?

Answer: A plain-text tabular format in which values are separated by delimiters, commonly commas.

Q. Does correlation mean causation?

Answer: No. Correlation describes association; it does not prove that one variable causes another.

Q. Does highest sales mean highest profit?

Answer: No. Profit depends on costs and discounts as well. This notebook sums Sales only.

Q. Why might region-wise sales be misleading alone?

Answer: It does not account for population, number of customers, costs, profit margins or time period. Additional context may be needed.

Q. What is split–apply–combine?

Answer: Split data into groups, apply a calculation to each group, and combine the group results.

Q. What is the difference between a Series and DataFrame?

Answer: Series is one-dimensional; DataFrame is two-dimensional and can contain multiple columns.

8. Code-level cross-questions and final revision

Likely modifications an examiner may ask

```
# Average sales by region
sales.groupby('Region')['Sales'].mean()

# Number of records in each region
sales.groupby('Region').size()

# Region-wise profit (not done in the original notebook)
sales.groupby('Region')['Profit'].sum()

# Sort regions by total sales, highest first
region_sales.sort_values(ascending=False)

# Check missing values
sales[['Region', 'Category', 'Sales']].isna().sum()

# Remove rows with missing required fields (one possible strategy)
sales = sales.dropna(subset=['Region', 'Category', 'Sales'])
```

These are explanatory examples, not extra lines present in your submitted notebook. Choose a missing-data strategy based on the task; do not automatically delete rows without checking how much data would be lost.

Potential examiner questions about limitations

Q. Does this code remove duplicate rows?

Answer: No. The notebook does not call drop_duplicates(), so duplicates are not explicitly removed.

Q. Does it handle all missing values?

Answer: No. It coerces invalid Sales values to NaN and unstack fills missing Region–Category combinations with zero. It does not explicitly inspect or remove all missing values.

Q. Does it convert Order Date to datetime?

Answer: No. Dates are not used in this experiment.

Q. Does it calculate top region automatically?

Answer: It displays the totals; the viewer can identify West as highest. The code does not explicitly call idxmax().

Q. What does idxmax() return?

Answer: The index label associated with the maximum value in a Series. For region_sales, it would identify the region with the highest total.

Q. Why use a stacked chart?

Answer: To compare total sales and the category composition of sales in each region.

Q. What if the CSV file is not found?

Answer: read_csv() raises a FileNotFoundError. Check the current working directory and the file path/name.

Q. Why might a CSV export include an index column?

Answer: to_csv() writes the index by default. Use index=False if you do not want the index written as a separate column.

Q. What is the purpose of display()?

Answer: It presents rich output in Jupyter. In a regular Python script, print() is often used for simple text output.

Q. Is plt.plot() required after the Pandas plot?

Answer: No, it is unnecessary for displaying this bar chart. In scripts, plt.show() is the conventional display call.

One-minute explanation to speak in the viva

“My experiment is Data Aggregation on retail sales data. First, I imported Pandas, NumPy, Matplotlib and Path. I loaded the CSV into a DataFrame and used head() to inspect the first five records. I checked the Region column's data type. Then

I converted Sales to numeric, and removed leading and trailing spaces from Region and Category. Next, I grouped the data by Region and calculated total Sales using sum(). I also grouped by Region and Category, summed Sales and used unstack() to make categories separate columns. I visualized the results using a normal bar chart and a stacked bar chart. Finally, I created an output folder and exported both summary tables as CSV files. The West region has the highest total sales in the displayed output."

Last-minute checklist

- Know the meaning of DataFrame, Series, groupby, sum and unstack.
- Explain why numeric conversion and strip() are done before grouping.
- Remember the highest region is West; lowest is South.
- Know that the notebook sums Sales, not Profit.
- Be honest: NumPy is imported but unused; duplicate removal is not implemented.
- Explain the difference between normal and stacked bar charts.

Study note: The explanations follow the uploaded notebook. Where a useful improvement or extra example is shown, it is labelled as an additional example rather than being presented as original notebook code.